

Introduction

The problem

Architecture

Trust and data

Business case

Operate and scale

Migration readiness assessment, run by a team of agents

An AI-native platform that turns raw discovery exports into a right-sizing map, 7R register, TCO model, gap register and wave plan — with architects approving every decision that carries risk.

Kamlesh

Founder, GSaraICloud · AI Architect portfolio

Built and running

Tested on estates of 1,000+ workloads

LangGraph StateGraph

Claude Sonnet

PostgreSQL + pgvector

Redis

FastAPI

React

AWS ECS

Jinja2 → DOCX/PDF

Most migration overruns are seeded in the assessment

Budgets and timelines are set from the assessment. When the inventory is incomplete, dependencies are guessed and TCO sits in an unchecked spreadsheet, the overrun is already baked in before the first server moves.

75%

of cloud migrations run over budget; more than a quarter overrun by 20% or more.

14%

average extra migration spend per year versus plan, attributed to coordination missteps.

38%

of companies saw migrations slip by more than one quarter.

Every migration programme starts here. AWS MAP makes the Assess phase — readiness assessment, portfolio discovery and a TCO business case — the entry point, and partners typically quote 2–6 weeks for it before larger estates add months of SME interviews.

Sources: McKinsey, “Cloud-migration opportunity: business value grows, but missteps abound” (survey of ~450 CIOs, 2020–21); AWS Prescriptive Guidance, Assess phase; AWS Marketplace MRA partner offerings.

How an MRA is done today: seven hand-offs, five teams, one very large spreadsheet

1	2	3	4	5	6	7
Collect	Consolidate	Right-size	Map dependencies	Classify 7R	Model TCO	Plan and report
Pull RVTools, discovery agent, Nessus, SolarWinds and Lansweeper exports. Six formats, different units.	Merge into a master workbook by hand. Duplicates, stale and missing hosts slip through.	Map vCPU and RAM to EC2 — often on allocated rather than used capacity.	Workshops with app owners; tribal knowledge fills gaps in flow data.	Debate Rehost vs Replatform vs Refactor app by app; rationale rarely written down.	Three-year on-prem vs AWS model in Excel; pricing and formulas copied between versions.	Sequence waves, write runbooks, assemble the MRA report and readout deck.
	Analysts	Cloud architects	Architects + app owners	Architects	Finance + architects	PMO

Infra team

No single source of truth

Data lives across dozens of workbooks:
MRA_v7_final_FINAL.xlsx.

Errors compound

A wrong row in step 2 flows silently into TCO and waves.

Decisions aren't traceable

Nobody can show why an app became Refactor.

Every client starts at zero

12-20+ people for 4-8 weeks, rebuilt per engagement.

What the platform automates, and what it deliberately doesn't

It covers the analytical core of the AWS MAP Assess phase end to end. The people-and-governance readiness conversation stays with humans — that's where judgment and trust are built.

Automated by agents

Ingestion and normalisation of seven discovery sources into one canonical schema

Right-sizing from p95 CPU and p99 memory, with a confidence score per workload

Dependency graph and application tiering from network flows

7R classification with a written rationale per application

TCO and ROI: 3-year on-prem vs AWS, RI/SP/On-Demand, MAP credits, sensitivity

Licence risk, EOL OS and gap registers (security, HA, DR, compliance)

Wave plan, runbooks, effort and schedule, compiled into DOCX/PDF, Excel and JIRA-ready outputs

Kept with humans

CAF readiness workshop across business, people and governance perspectives

Business context: criticality, owners, change freezes, RTO/RPO targets

Approval gates on low-confidence sizing, 7R exceptions and TCO assumptions

Final sign-off of the wave plan and the executive readout

Architects move from assembling data to reviewing decisions — which is what clients actually pay for.

Architecture: hub-and-spoke, two waves of specialist agents

Layer 1
Ingest and normalise

Sources
RVTools · AWS ADS · Migration Evaluator
· Nessus · SolarWinds · Lansweeper ·
scripts

Normaliser
Schema validation, dedup, unit
normalisation, canonical JSON

Staging store
PostgreSQL + pgvector: inventory,
utilisation, flows

Layer 2
Hub

Coordinator agent — LangGraph StateGraph
Execution order · conflict resolution · human-in-the-loop escalation

Shared context
Redis — all agents read/write

Layer 3
Wave 1 spokes

Right-sizing
p95 CPU · p99 mem → EC2 + confidence

Dependency
Flow analysis → directed graph, tiers

7R bucketing
Scoring matrix → R-class + rationale

Layer 4
Wave 2 spokes,
consume Wave 1

TCO calc
3-yr model, RI/SP, MAP credits, scenarios

Assessment
OS/licence/DB, Oracle risk, EOL
2008/2012

Gap analysis
Security/HA/DR, compliance, P1-P3

Layer 5
Wave planning

Dependency sequencer
Topological sort → move groups

Priority scorer
5-dimension MPS → wave order

Migration factory
Runbooks, effort, schedule

Layer 6
Outputs

MRA report
DOCX / PDF

Wave plan
Excel / CSV

TCO model
Excel

7R register
JSON / CSV

Gap register
DOCX / JIRA

Coordinator monitors every stage

- Introduction
- The problem
- Architecture**
- Trust and data
- Business case
- Operate and scale

ADR-001: Orchestrate specialist agents through a central coordinator

Status

Accepted

Context

MRA is a chain of dependent judgments. TCO is meaningless without right-sizing; wave order needs the dependency graph and 7R result. Agents disagree — 7R may say Rehost while Assessment flags an EOL Windows 2008 OS.

Decision

One coordinator owns state, ordering, conflict resolution and escalation. Spokes are narrow, stateless experts that read from and write to shared context.

Consequence

The hub is a bottleneck and single point of failure — mitigated with LangGraph checkpointing, idempotent spokes and per-node retries.

Option considered	Strength	Why not / why
Single “do-everything” agent	Simplest to build	Context overload on 1,000+ workloads; impossible to evaluate one capability in isolation
Linear prompt chain	Deterministic, easy to trace	No place to resolve conflicts between findings; one failure stops the chain
Peer-to-peer swarm	Flexible, emergent	Non-deterministic paths are hard to audit — clients and auditors need to see why a decision was made
Hub-and-spoke	Controlled order, parallel spokes, one audit trail	Chosen: gives explicit ordering, parallelism inside a wave, a single place for human escalation, and per-agent evaluation

Supporting decisions and their rationale

ADR-002

Two waves with an explicit contract

Wave 2 only starts when Wave 1 results are written to shared context. A TCO built before right-sizing is fiction, so the graph enforces it.

ADR-005

Two stores, two jobs

Redis holds fast, per-run working context. PostgreSQL + pgvector holds durable inventory and lets agents find similar, previously classified applications.

ADR-003

Code does the maths, the LLM does judgment

Percentiles, pricing, TCO formulas and topological sort are deterministic code. The model classifies, explains and flags anomalies — so numbers are reproducible.

ADR-006

LangGraph over a custom loop

An explicit state graph gives checkpoint and resume, interrupts for human approval, and a replayable execution trace for every run.

ADR-004

Canonical schema at the edge

Agents never see vendor formats. Supporting a new discovery tool — a Device42 or ServiceNow CMDB export — means writing one parser, not touching an agent.

ADR-007

Model behind configuration

The model ID is configuration, not code, so it can be upgraded, evaluated against the regression set, or served in-region through Amazon Bedrock for data residency.

Agent catalogue: what each one decides, and who checks it

Agent	Consumes	Decides	Produces	Human checkpoint
Right-sizing	Utilisation time series	EC2 family and size from p95 CPU / p99 memory	Instance map + confidence	Review low-confidence items
Dependency	NetFlow, agent connections	Edges, tiers, application groups	Dependency graph JSON	Confirm app boundaries with owners
7R bucketing	Inventory, sizing, graph	Retire / Retain / Rehost / Replatform / Refactor...	7R register + rationale	Approve Retire and Refactor calls
TCO calc	Sizing, 7R, on-prem costs	Pricing model, MAP credits, scenarios	Cost delta + ROI	Sign off assumptions with finance
Assessment	OS, DB, licence data	Oracle licence risk, EOL OS, RDS vs self-managed	Licence risk register	Escalates conflicts to hub
Gap analysis	All findings + Nessus	Security, HA, DR and compliance gaps, P1-P3	Gap remediation plan	Sign off P1 gaps with security lead
Wave planning	Graph + 5-dimension MPS	Move groups and wave order	Waves, runbooks, schedule	Approve plan with PMO and client

Data strategy: the platform is only as good as the inventory

The biggest risk isn't the model — it's garbage in. So the data contract is agreed before the engagement starts, and the pipeline refuses to hide gaps.

Minimum data contract

Data	Source	Need
Server inventory	RVTools, CMDB, ADS	Required
Utilisation, incl. month-end peaks	Discovery agents, Migration Evaluator	Required
Network flows	SolarWinds NetFlow, ADS	Strongly advised
Software and licences	Lansweeper, scripts	Strongly advised
Vulnerabilities	Nessus	Recommended
Business context	Owner interviews	Recommended

Quality gates before any agent runs

- Schema validation** — reject malformed rows with a reason, don't drop them silently
- Cross-source deduplication** — one server, one record, regardless of which tool found it
- Unit normalisation** — vCPU, GB vs GiB, time zones on utilisation
- Reconciliation** — counts checked against the client CMDB; missing workloads listed
- Completeness score per workload** — carried into every downstream confidence score and shown in the report

- Introduction
- The problem
- Architecture
- Trust and data**
- Business case
- Operate and scale

Security and governance: discovery data is an attacker's map

Hostnames, IPs, software versions and open vulnerabilities for an entire estate. The design treats it as the most sensitive data the consultancy holds.

Access	Isolation	AI-specific	Governance
Role-based access, limited to the named engagement team	Separate schema and key namespace per client	Hostnames and IPs masked before prompts	Every agent decision logged with inputs and rationale
Roles: lead, architect, finance viewer, client read-only	Encryption at rest with KMS, TLS in transit	In-region model serving option via Bedrock	Retention and purge at engagement close
SSO with MFA	Deployed in ap-south-1 (Mumbai)	Structured, validated outputs only	Aligned to India's DPDP Act and sector rules for BFSI clients

Human oversight: automation removes the drudgery, not the accountability

The manual process fails because people spend their time copying data between spreadsheets. Here the agents do the assembly, and people are pulled in at four gates where a wrong call is expensive.

Ingest Normalise, dedupe, reconcile	Gate 1 Data sign-off: gaps and missing workloads accepted	Wave 1 Sizing, dependencies, 7R	Gate 2 Low-confidence sizing and 7R exceptions reviewed	Wave 2 TCO, licences, gaps	Gate 3 TCO assumptions and P1 gaps signed off	Plan Waves, runbooks	Gate 4 Wave plan approved with client
---	--	---	--	--	---	-----------------------------------	---

Example escalation

7R agent proposes *Rehost* for a billing app. Assessment agent flags Windows Server 2008 and an Oracle licence tied to physical cores. The coordinator doesn't pick a winner — it pauses the graph and routes both findings to an architect.

Why this matters to clients

Every recommendation in the report can be traced to source rows, the agent that made it, and the person who approved it. That's what makes an AI-produced assessment defensible in a steering committee.

Accuracy: measured per agent against expert judgment

95%

agreement with architect-validated results on the internal evaluation set.

Most MRA questions have no single ground truth — two good architects can disagree on Replatform vs Refactor. So accuracy is reported as agreement with expert review, broken down by agent, and re-run on every prompt or model change.

Agent	Metric	Compared against
Right-sizing	Same EC2 family, size within one step	Architect sizing on sampled workloads
Dependency	Edge precision and recall	Flows confirmed in owner workshops
7R bucketing	Class agreement; rationale accepted	Architect-labelled register
TCO calc	Variance vs independent calculation	Hand-built model + AWS pricing
Assessment	EOL / licence flag recall	Known EOL and Oracle hosts
Report	Every number traceable to a source row	Automated fact-check pass

Real-world use: a 487-workload Indian mid-market estate

The first client assessment was built the traditional way — Excel, PowerPoint and Word. Running a structured validation over it is what shaped the platform's guardrails.

34

workloads missing from the consolidated inventory — found only by reconciling sources against each other.

1

TCO formula error propagating through the business case — now impossible because the maths is code, not cells.

32%

credible savings after recalibration, replacing implausible 95-97% claims. AWS reports a 31% average infrastructure saving for migrated legacy apps.

The most valuable component turned out to be the one that checks the others.

Client anonymised. Benchmark: AWS Migration Acceleration Program page. A reusable 15-prompt QA library came out of this validation and now runs as the report fact-check.

Business case: from a staffing exercise to a review exercise

	Manual MRA today	With the platform
Team for a large estate	20+ across infra, architecture, finance, security, PMO	3-4 architects plus the platform
Analysis turnaround	Weeks of consolidation before analysis starts	Agent run in hours; time goes to SME review
Where the data lives	Dozens of workbook versions	One versioned store per client
Decision traceability	In people's heads	Rationale, source rows and approver per decision
Re-running a scenario	Days of spreadsheet rework	Change an assumption, re-run Wave 2
Next client	Start from zero	Same pattern, new data

For the consultancy

More assessments with the same bench, and a repeatable IP asset instead of hero effort.

For the client

A faster, cheaper and more defensible business case — and fewer surprises in the migration budget.

For AWS programmes

Assess-phase outputs ready sooner, so Mobilize and MAP funding decisions start earlier.

ROI, with the assumptions on the table

Change any input — the numbers are yours to challenge. Defaults are a conservative mid-market case, not the best case.

Manual assessment

People on the assessment	12
Working weeks	6

With the platform

People on the assessment	4
Working weeks	3

Economics

Blended day rate (₹)	20000
Assessments per year	4
Platform cost per year (₹ lakh)	80

200%

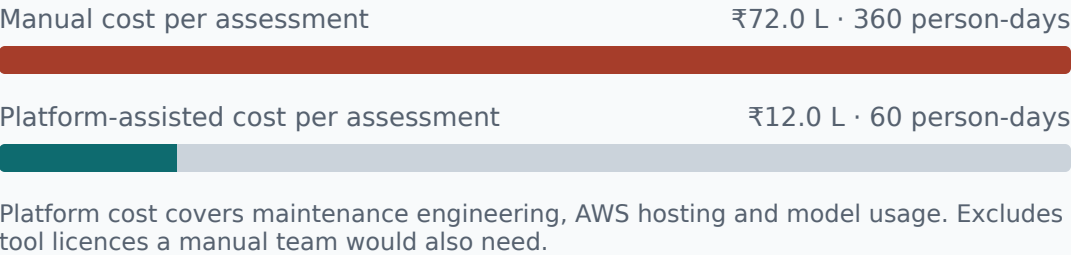
annual return on the platform's running cost: (effort saved – platform cost) ÷ platform cost

83%

effort reduction per assessment

₹1.60 Cr

net annual saving after platform cost



Measurable outcomes tracked for every engagement

Because every run is recorded, these are measured, not estimated after the fact.

KPI	How it's measured	Target
Assessment cycle time	Data received to readout delivered	60–75% shorter
Effort per 100 workloads	Person-days logged against the engagement	70%+ lower
Data defects caught before the report	Reconciliation and validation findings	All listed; none silently dropped
Number traceability	Report figures linked to source rows	100%
Forecast accuracy	TCO forecast vs actual AWS bill six months after cutover	Within $\pm 15\%$
Client rework	Report revision cycles after first readout	One or fewer
Onboarding a new client	Kickoff to first agent run	Days, not weeks

Risks and how the design contains them

Risk	Impact	Mitigation
Incomplete or incorrect input data	High	Data contract, reconciliation, completeness scores that lower downstream confidence; Gate 1 sign-off
Hallucinated rationale or findings	High	Schema-validated outputs, rationale must cite source rows, deterministic maths, report fact-check
Over-optimistic savings	High	Sensitivity scenarios, benchmark guardrail flags savings far outside industry ranges, finance sign-off
Client data exposure	High	Per-client isolation, RBAC, masking before prompts, in-region hosting, purge at close
Stale pricing or model drift	Medium	Pricing refreshed from AWS price data per run; evaluation regression on every model or prompt change
Automation bias	Medium	Confidence shown on every item; mandatory gates cannot be skipped
Estates beyond tested scale	Low	Batch by application group, parallel spokes, horizontal worker scaling

Scale and cost: tested at 1,000+ workloads, built for many clients

Why it scales

Per-server work is vectorised code — percentiles, sizing and pricing don't need an LLM call per server

LLM calls scale with application groups, not server count — rationale is written per app, not per VM

Spokes run in parallel inside each wave

Stateless FastAPI workers on ECS scale horizontally; state lives in Redis and Postgres

Checkpointed runs resume from the failed node instead of restarting

Cost at scale

Compute and storage — a few small containers plus managed Postgres and Redis, shared across clients

Model usage — grows with application count; tracked per run from traces, so each engagement has a known cost

Multi-tenant history — each client's MRA journey, TCO versions, 7R register and wave plans kept side by side

Marginal cost of the next client is data onboarding and review time, not a new team

- Introduction
- The problem
- Architecture
- Trust and data
- Business case
- Operate and scale**

Why build this when AWS Transform exists?

AWS Transform now offers agentic assessments with what-if TCO scenarios in minutes, at no cost for core migrations. It's a strong baseline — and the right thing to compare against.

	AWS Transform assessments	This platform
Primary job	Fast AWS business case and migration execution	The consultant's full MRA deliverable set
Inputs	RVTools, CMDB, AWS discovery, third-party tools	Same, plus vulnerability scans and NetFlow for gap analysis
Security and compliance gaps	Not the focus	P1-P3 gap register mapped to client compliance needs
Licence risk	Licensing analysis in the TCO	Oracle and EOL risk register with remediation paths
Methodology	AWS's	The consultancy's own, encoded and versioned
Portability	AWS-specific	Pattern reusable for other targets and clients

Positioning: complementary. Transform outputs can be ingested as another source; the value added here is the governed judgment layer on top.

- Introduction
- The problem
- Architecture
- Trust and data
- Business case
- Operate and scale**

Implementation approach

Platform delivery path

Mock mode	Local development with synthetic estates and stubbed model calls — fast, free iteration on agent logic
Containerised	Docker Compose with Postgres, Redis, API and dashboard — the full system on one laptop
Cloud	Containers on AWS in ap-south-1, managed data stores, per-client isolation
Evaluation first	Golden estate and per-agent metrics added before new features

Engagement playbook

Before kickoff	Agree data contract; start discovery collection to capture peaks
Day 1	Ingest, reconcile, Gate 1 data sign-off
Day 2	Wave 1 and Wave 2 agent runs; Gate 2 and 3 reviews start
Week 1	SME and app-owner review of flagged items; scenario re-runs
Week 2	Gate 4 wave plan approval; executive readout

Lessons learned and best practices

Never let the model do arithmetic

The worst errors were numeric: a broken TCO formula and 95-97% savings claims. Numbers belong in tested code.

Reconcile before you analyse

34 missing workloads were invisible until sources were checked against each other. Reconciliation is now a gate, not a task.

Build a sanity layer

Results far outside industry benchmarks — like savings well above the ~31% AWS average — trigger review automatically.

Evals before features

A golden estate and per-agent metrics made every prompt and model change safe to ship.

The schema is the reuse boundary

Reuse across clients came from the canonical data model, not from clever prompts.

Design the gates in

Human checkpoints bolted on later get skipped. Built into the graph, they become part of the workflow clients trust.

Also: fail fast on configuration — a single inline comment in an environment file once broke typed settings validation at start-up.

What's next, and what this shows

Roadmap

In-region model serving through Amazon Bedrock for regulated clients

Readiness questionnaire agent to cover the CAF perspectives

Feedback loop from actual post-migration bills into the TCO agent

MCP connectors to pull discovery data directly from source tools

Governance framework integration for responsible-AI controls across agents

What this project demonstrates

Turning a 20-year-old consulting process into a **governed multi-agent system**

Architecture decisions with trade-offs, not just a working demo

Evaluation and guardrails driven by real failures on a real estate

A **business case** expressed in the client's and the CFO's terms

Agents do the assembly. Architects own the decisions.